

Project: Async Task Runner with Event Loop Visualizer

Project Scenario

Tum ek developer tools startup mein kaam kar rahe ho.

Task: Ek async task runner banana hai jo: callbacks support kare,
callback hell se bachne ke liye internally named functions use kare,
setTimeout/setInterval based scheduling kare,
debounce/throttle utilities provide kare,
aur ek browser console-based 'event loop visualizer' banao jo queue states dikhaye.

Part 1 — Task Queue (Callback-based)

Ek callback-based task queue jo tasks ko schedule aur execute karta hai.

Task Queue with Concurrency

```
// taskQueue.js
function createTaskQueue(concurrency = 1) {
  const queue = [];
  let running = 0;
  function run() {
    if (running >= concurrency || queue.length === 0) return;
    running++;
    const { task, done } = queue.shift();
    const start = Date.now();
    try {
      task((err, result) => {
        running--;
        const duration = Date.now() - start;
        if (err) {
          done(err, null, duration);
        } else {
          done(null, result, duration);
        }
      });
    } catch (syncErr) {
      running--;
      done(syncErr, null, Date.now() - start);
    }
    run(); // pick next task from queue
  }
  return {
    queue,
    run
  };
}
```

```

}
return {
  add(task, callback) {
    queue.push({ task, done: callback });
    run();
  },
  get size() { return queue.length; },
  get active() { return running; },
  get pending() { return queue.length; },
};
}

// Usage
const q = createTaskQueue(2); // 2 concurrent tasks
function simulateWork(name, ms, shouldFail) {
  return (callback) => {
    console.log(` Starting: ${name}`);
    setTimeout(() => {
      if (shouldFail) callback(new Error(`${name} failed`));
      else callback(null, `${name} done`);
    }, ms);
  };
}

['Task-A', 'Task-B', 'Task-C', 'Task-D'].forEach((name, i) => {
  q.add(simulateWork(name, (i+1)*300, name==='Task-C'), (err, result, ms) => {
    if (err) console.log(` ✕ ${err.message} (${ms}ms)`);
    else console.log(` ✓ ${result} (${ms}ms)`);
    console.log(` Queue: active=${q.active} pending=${q.pending}`);
  });
});

```

Part 2 — Scheduler (setTimeout/setInterval based)

Cron-like scheduler using recursive setTimeout — drift-free.

Drift-Free Scheduler

```

// scheduler.js
function createScheduler() {
  const jobs = new Map();
  let jobId = 0;

  function scheduleOnce(fn, delay, label = '') {
    const id = ++jobId;
    const timerId = setTimeout(() => {
      jobs.delete(id);
      try { fn(); }
      catch(e) { console.error(`Job '${label}' error:`, e.message); }
    }, delay);
    jobs.set(id, { timerId, type: 'once', label, delay });
    return id;
  }
}

```

```

function scheduleRepeat(fn, interval, label = '') {
  const id = ++jobId;
  let expected = Date.now() + interval;
  function tick() {
    if (!jobs.has(id)) return; // cancelled
    try { fn(); }
    catch(e) { console.error(`Job '${label}' error:`, e.message); }
    const drift = Date.now() - expected;
    expected += interval;
    const timerId = setTimeout(tick, Math.max(0, interval - drift));
    jobs.get(id).timerId = timerId; // update ref
  }
  const timerId = setTimeout(tick, interval);
  jobs.set(id, { timerId, type: 'repeat', label, interval });
  return id;
}

function cancel(id) {
  const job = jobs.get(id);
  if (!job) return false;
  clearTimeout(job.timerId);
  jobs.delete(id);
  return true;
}

function listJobs() {
  return [...jobs.entries()].map(([id, j]) =>
    `[${id}] ${j.type} '${j.label}' every ${j.interval} || j.delay}ms`
  );
}

return { scheduleOnce, scheduleRepeat, cancel, listJobs };
}

const scheduler = createScheduler();
const heartbeatId = scheduler.scheduleRepeat(
  () => console.log('Heartbeat', new Date().toLocaleTimeString()),
  1000, 'heartbeat'
);
scheduler.scheduleOnce(
  () => { scheduler.cancel(heartbeatId); console.log('Heartbeat stopped'); },
  5000, 'stop-heartbeat'
);
console.log('Jobs:', scheduler.listJobs());

```

Part 3 — Debounce + Throttle Utilities

Production Debounce & Throttle with cancel/flush

```

// utils.js – production-grade debounce/throttle

export function debounce(fn, delay, { leading = false } = {}) {
  let timerId = null;

```

```

let lastCallTime = 0;
function debounced(...args) {
  const now = Date.now();
  const shouldCallNow = leading && (now - lastCallTime > delay);
  clearTimeout(timerId);
  lastCallTime = now;
  if (shouldCallNow) {
    fn.apply(this, args);
  } else {
    timerId = setTimeout(() => {
      if (!leading) fn.apply(this, args);
      timerId = null;
    }, delay);
  }
}
debounced.cancel = () => {
  clearTimeout(timerId);
  timerId = null;
};
return debounced;
}

export function throttle(fn, limit) {
  let lastCall = 0;
  let timerId = null;
  function throttled(...args) {
    const now = Date.now();
    const remaining = limit - (now - lastCall);
    if (remaining <= 0) {
      if (timerId) { clearTimeout(timerId); timerId = null; }
      lastCall = now;
      fn.apply(this, args);
    } else if (!timerId) {
      timerId = setTimeout(() => {
        lastCall = Date.now();
        timerId = null;
        fn.apply(this, args);
      }, remaining);
    }
  }
  throttled.cancel = () => {
    clearTimeout(timerId); timerId = null; lastCall = 0;
  };
  throttled.flush = () => {
    clearTimeout(timerId); timerId = null; fn();
  };
  return throttled;
}

```

Part 4 — Console Event Loop Visualizer

Browser console mein queues ka state print karo — learning tool.

Console Visualizer — Teaching Tool

```
// visualizer.js — Event Loop State Logger
const EventLoopViz = (() => {
  const callStack = [];
  const microtasks = [];
  const macrotasks = [];
  let stepNum = 0;

  function log(phase) {
    console.log(`%c[Step ${++stepNum}] ${phase}`, 'color:#7F77DD;font-weight:bold');
    console.log(` Call Stack: [${callStack.join(' → ') || 'empty'}]`);
    console.log(` Microtasks: [${microtasks.join(', ') || 'empty'}]`);
    console.log(` Macrotasks: [${macrotasks.join(', ') || 'empty'}]`);
    console.log('');
  }

  return {
    push(label) { callStack.push(label); log(`PUSH: ${label}`); },
    pop() { const l = callStack.pop(); log(`POP: ${l}`); },
    addMicrotask(l) { microtasks.push(l); log(`Microtask queued: ${l}`); },
    runMicrotask() { const l = microtasks.shift(); log(`Microtask run: ${l}`); },
    addMacrotask(l) { macrotasks.push(l); log(`Macrotask queued: ${l}`); },
    runMacrotask() { const l = macrotasks.shift(); log(`Macrotask run: ${l}`); },
    reset() { callStack.length=0; microtasks.length=0; macrotasks.length=0; stepNum=0; }
  };
})();

// Simulate: console.log('A'); Promise.resolve().then(()=>log('B'));
setTimeout(()=>log('C'),0);
EventLoopViz.push('main script');
EventLoopViz.push('console.log A');
EventLoopViz.pop();
EventLoopViz.addMicrotask('Promise.then -> B');
EventLoopViz.addMacrotask('setTimeout -> C');
EventLoopViz.pop(); // main script done
EventLoopViz.runMicrotask();
EventLoopViz.runMacrotask();
```

Bonus Challenges

1. Parallel task queue: `createTaskQueue(concurrency=3)` — 3 tasks run simultaneously.
2. Retry scheduler: agar task fail ho to exponential backoff ke saath retry karo (max 3 times).
3. Debounce leading edge bhi implement karo (option flag `{ leading: true }`).
4. Task prioritization: queue mein 'high'/'low' priority tasks — high wale pehle run hon.
5. Timeline logger: har task ke start/end time track karo, Gantt-chart style timeline print karo.